SystemVerilog Phase 2 – Kiana Jafari:

This project mainly focuses on implementing a shallow neural network from scratch in Python (using NumPy for implementation), and inferring its approximate performance in SystemVerilog. The dataset used for this project is called the Wisconsin Breast Cancer Dataset (WBCD) from the `sklearn.datasets` package.

---

1. Data Analysis:

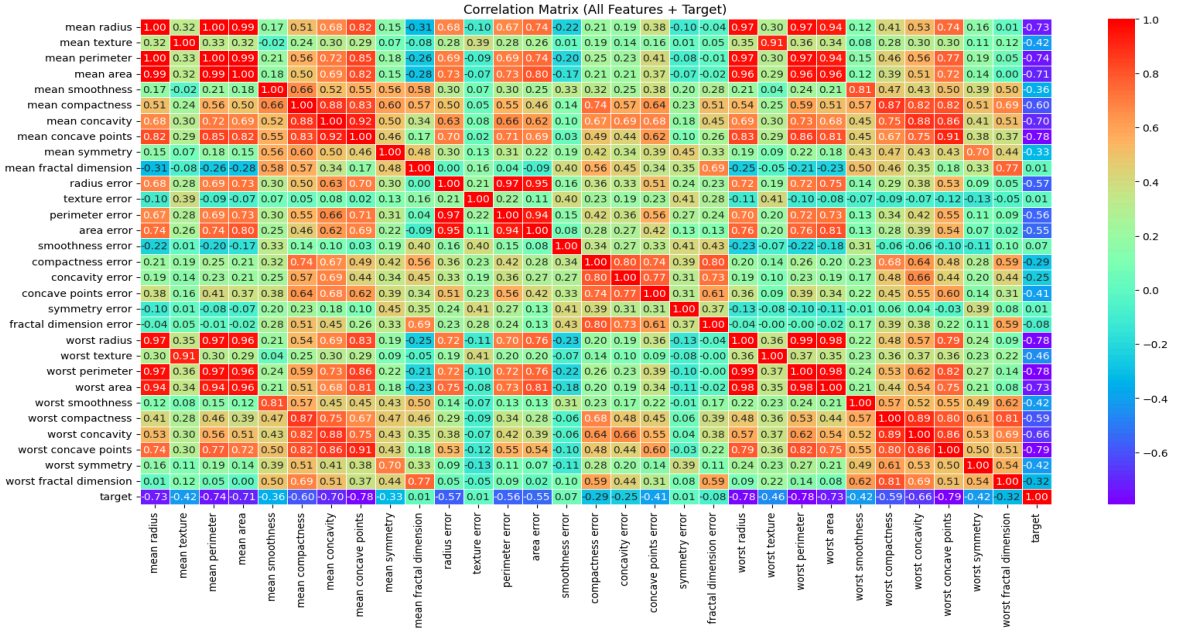
1. **Dataset:** The first step involves loading in and analyzing the data using `Pandas`, `NumPy`, `Matplotlib`, and `Seaborn`. The original data has about 30 features and 569 records (examples).

A sample of the data is shown below:

mean radius	mean texture	worst smoothness	worst texture	worst perimeter
17.99	10.38	0.1622	17.33	184.60
20.57	17.77	0.1238	23.41	158.80
19.69	21.25	0.1444	25.53	152.50

2. **Descriptive Summary:** The number of null rows has been checked using data.info(). It's worth mentioning that no NaN was found. Also, a descriptive summary showed the distribution of the features using data.describe() method.

3. **Data Visualization:** Since 30 features are too much for our small network, we only utilized the two most correlated features as the input. We're essentially looking for the two features that not only have a low correlation with each other, but also a good ratio with the target. A heatmap has been plotted to show the correlation between features and the target. (Fig.)



4. **Data Selection:** Since choosing the two most correlated features based on only a Heatmap would be a difficult choice, we also utilized the quantitative metrics, **Pearson correlation** (from the `SciPy` package), and **feature-pair scoring** method to make sure the selected features are scientifically reasonable.

The feature-pair scoring has been computed using the following formula:

$$score = |corr(A, y)| \, |corr(B, y)| \, |(1 - |corr(A, B)|)$$

Where:

- corr(A, y): correlation between feature A and target
- corr(B, y): correlation between feature B and target
- corr(A, B): correlation between the two features
- λ: penalty strength for redundancy

Top 3 features with the lowest multicollinearity and highest Pearson correlation:

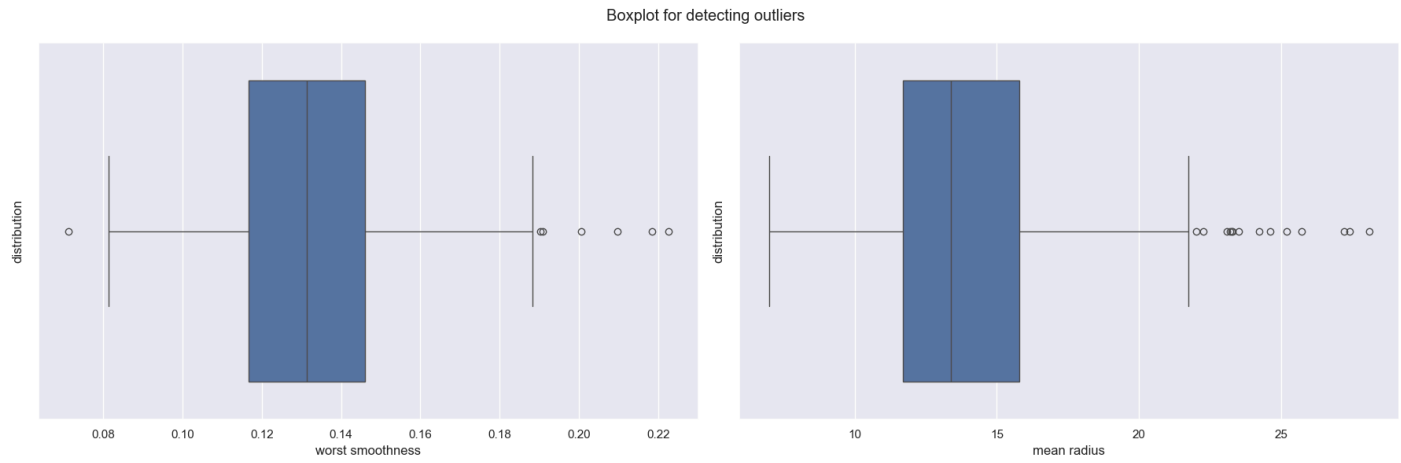
feature_1	feature_2	target_corr_1	target_corr_2	feature_corr	score
mean radius	worst smoothness	0.730029	0.421465	0.119616	0.270878
mean perimeter	worst smoothness	0.742636	0.421465	0.150549	0.265874
mean area	worst smoothness	0.708984	0.421465	0.123523	0.261902

"worst smoothness" appears alongside other features in all three sets. The pair of features in the first row has the best performance, since the score is higher and the feature_corr is lower. The higher the score, the lower the collinearity between features. It is worth mentioning that for further analysis, training the network with all three pairs might end up in a more accurate result. This experiment is especially worth it in selecting the best pair of features. As an example, *mean perimeter* in the second pair has a higher correlation than the *mean radius* in the first pair. But we should also take into consideration that the higher the correlation of the features, the lower the final score. Therefore, selecting the best pair of features is actually a trade-off that should be handled in further analysis.

4.1. **Data Preprocessing:** By using further visualization techniques like box plots, we discovered that the selected features include outliers, which are data points that fall outside a defined range. Since Neural Networks are sensitive to outliers, we conducted further analysis and implemented various functions to detect them. Interestingly, these outliers reveal a significant relationship between the tumors and the associated target, which is why imputing them might not be a good idea. Because, as extreme as they might be, they are sensitive and can add useful info.

A box plot is shown below:

(Fig.)



5. **Feature Scaling:** We use scaling to address the differences in variable ranges shown in the figure. Methods like `MinMaxScaler()` are applied to the features after splitting them into train-test sets, ensuring they fall within the range of [-1, 1].

Info	**worst smoothness**	**mean radius**
count	569	569
mean	0.132369	14.127292
std	0.022832	3.524049
min	0.071170	6.981000
25%	0.116600	11.700000
50%	0.131300	13.370000
75%	0.146000	15.780000
max	0.222600	28.110000

---

2. Model Implementation:

1. **Data Splitting:** The process of training a model begins by splitting the clean data into two sets: training and testing. If there is sufficient data, it is also beneficial to create a validation set. Our analysis revealed a class imbalance in the target variable; therefore, we set the `stratify` parameter to `y` (the target). Notably, 10% of the entire dataset was reserved for the test set, while the remaining 90% was allocated for training.

2. **Data Preprocessing:** The network needs scaled, preprocessed training-testing sets before being fed. X_train should be fit and transformed, while X_test should only be transformed. A data transpose is also required for the X values for further matrix multiplication computations. Target values should be one-hot encoded.

3. **Neural Network:**

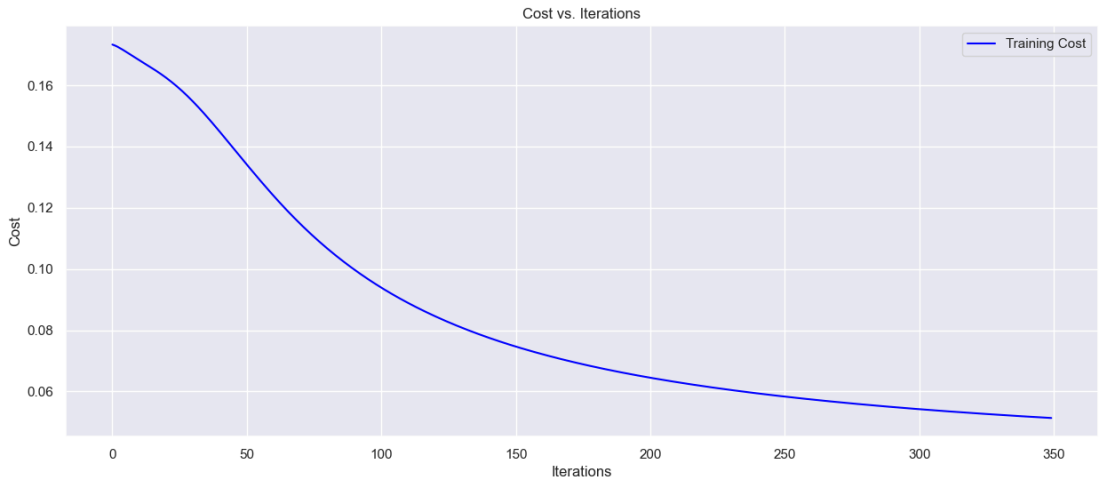
a. **Activation Functions**: The two activation functions used in this project are mainly `ReLU` (also its derivative for backpropagation) and `Softmax`.

b. **Methods:** Each of the steps in a neural network, from initializing the parameters to updating them, should be written as a single method, each of them responsible for a certain task. This is known as the Single Responsibility Principle (or SRP). Please note that the network is implemented from scratch (using NumPy and pure Python), since it is easier to retrieve and store the weights/biases for smaller datasets. Methods include: *`initialize_parameters`, `forward_propagation`, `compute_cost`, `backward_propagation`, `initialize_adam`, `update_parameters`*

c. **Class Neural Network:** The network is written using a class and has methods that are in a separate file called *`methods.py`*. The general purpose of this class is to train the network and thus, get the best set of parameters (i.e., updated parameters retrieved from the Gradient Descent). The hyperparameters like `learning_rate`, `decay_rate`, `penalty`, and `epochs` are approximated using `GridSearchCV`.

d. **Result:** The training resulted in 93.75% accuracy on the training set and 92.98% on the testing set. The network successfully predicted 53/57 data points of the test set. Indices with the wrong predictions are array([2, 5, 40, 56]). These might be the indices of the outliers we detected earlier.

A visual representation of the training's cost function has been provided below:



e. **Evaluation Metrics:** Below, some useful evaluation metrics have been provided that show the network performance. These include: *classification report & confusion matrix*

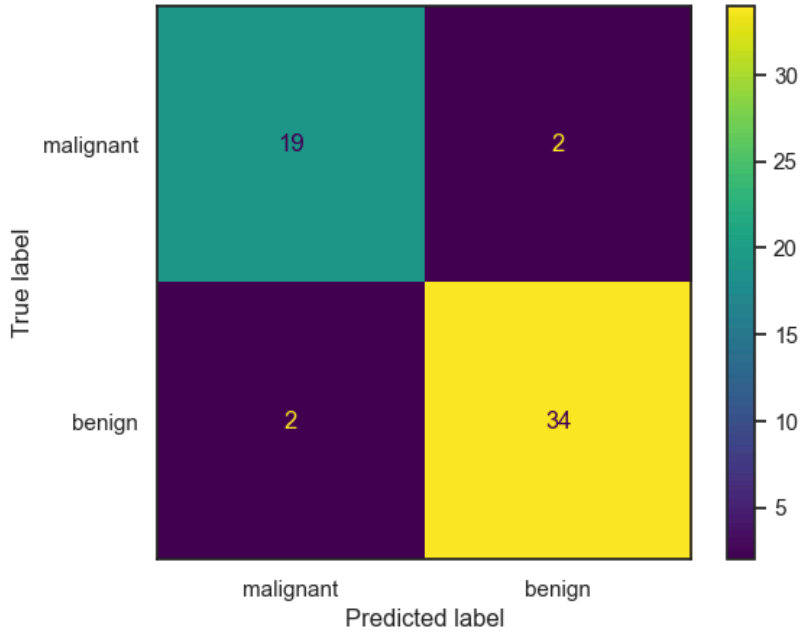
The metrics indicate that the network was certain about benign test cases and did a better job in predicting those data points.

Classification Report:

Info	precision	recall	f1-score	support
malignant	0.90	0.90	0.90	21
benign	0.94	0.94	0.94	36
accuracy			0.93	57
macro avg	0.92	0.92	0.92	57
weighted avg	0.93	0.93	0.93	57

Confusion Matrix:

[[19 2]
[2 34]]



11 test data points are shown below:

Sample 0: Pred = 0, True = 0
Sample 1: Pred = 1, True = 1
Sample 2: Pred = 1, True = 0
Sample 3: Pred = 0, True = 0
Sample 4: Pred = 0, True = 0
Sample 5: Pred = 0, True = 1
Sample 6: Pred = 0, True = 0
Sample 7: Pred = 1, True = 1
Sample 8: Pred = 1, True = 1
Sample 9: Pred = 1, True = 1
Sample 10: Pred = 1, True = 1

f. **Parameters:** Since our focus is mainly on designing a hardware-based simulated network in SystemVerilog, for the sake of simplicity, the weights/biases are converted to **fixed-point** rather than **floating-point**. Note that the parameters, weighted sum, and the activations are all required for input quantization.

a. An overview of floating-point parameters:

First-layer weight matrix (W_1)

Hidden Neuron	worst smoothness	mean radius
h_1	0.05276146	0.04545019
h_2	0.87860578	1.4938967
h_3	0.04193307	0.03875046
h_4	0.84913852	1.46507294

First-layer bias vector (b_1)

Hidden Neuron	Bias
h_1	-0.0500322
h_2	0.93571265
h_3	-0.05000003
h_4	0.90332097

Second-layer weight matrix (W_2)

Output Neuron	h_1	h_2	h_3	h_4
O_1	-0.0343271	1.42355861	-0.05331349	1.46707236
O_2	0.01890401	-1.42651996	0.0436103	-1.45686225

Second-layer bias vector (b_2)

Output Neuron	Bias
O_1	-1.20539659
O_2	1.20539659

b. Fixed-point parameters:

First-layer weight matrix (W_1)

Hidden Neuron	worst smoothness	mean radius
h_1	216	186
h_2	3599	6119
h_3	172	159
h_4	3478	6001

First-layer bias vector (b_1)

Hidden Neuron	Bias
h_1	-205
h_2	3833
h_3	-205
h_4	3700

Second-layer weight matrix (W_2)

Output Neuron	h_1	h_2	h_3	h_4
O_1	-141	5831	-218	6009
O_2	77	-5843	179	-5967

Second-layer bias vector (b_2)

Output Neuron	Bias
O_1	-4937
O_2	4937

3. Parameters (weights/biases):

To make predictions and run inferences simpler in the hardware, the parameters are converted to fixed-point using **Q3.12** (scale = 4096). We use Q3.12 because it provides a good balance between:

- enough range (avoid overflow)
- enough precision (small quantization error)

This can be done using the following steps for computation:

- The largest magnitude during inference is found from the network activations/logits
- $M = max(|x|) = max(|Z2|) \approx 5.115514836880694$
- $2^m > M$
- $2^m > 5.115514836880694$, $log_2^{(5.115514836880694)} = 2.355$, $M = \lceil 2.355 \rceil = 3$
- $n = 15 - m = 15 - 3 => n = 12$
- Qm.n ⟶ Q3.12

Note that *m* is the number of integer magnitude bits.

Q3.12 supports the range [-8, 8), while still preserving good precision using 11 fractional bits.

How are floating-point values converted to fixed-point:

- Quantization scale: $2^{12} = 4096$
- $X_{fixed} = round(x * 4096)$

Quantized inference achieved the same testing accuracy (92.98%) as floating-point inference, validating the suitability of the selected fixed-point representation.